

به نام خدا

گزارش تمرین کامپیوتری سوم

سیگنال‌ها و سیستم‌ها-دکتر اخوان

امیرمحمد میرحسینی ۸۱۰۱۰۲۶۰۱

بخش اول:

توضیح فایل اصلی:

در اینجا یک آرایه سلولی به نام Mapset تعریف می‌کنیم که دو ردیف و ۳۲ ستون دارد. این آرایه برای نگهداری حروف و معادل‌های باینری‌شون استفاده میشود. در این حلقه، حروف الفبا (از 'a' تا 'z' به همراه معادل باینری ۵ بیتی‌شون در Mapset ذخیره می‌شن `dec2bin(i, 5)`. عدد آرو به باینری تبدیل می‌کنه و ۵ رقم رو برای نمایش در نظر می‌گیره.

بعد از آن تصویر خاکستری شده رو برای تقسیم به بلوک‌های ۶x۶ محاسبه میشود و تصویر به اندازه جدید تغییر می‌کنه. متن مخفی که می‌خواهیم در تصویر کدگذاری کنیم، در اینجا به عنوان hidden تعریف میشود. بعد از آن تصویر اصلی و تصویر کدگذاری‌شده رو نمایش می‌دیم. در نهایت، تابع decoding برای استخراج پیام مخفی از تصویر کدگذاری‌شده فراخوانی میشود و نتیجه نمایش داده میشود.

توضیح فانکشن کدینگ:

ابتدا در اینجا اندازه تصویر و تعداد بلوک‌ها محاسبه میشود و طول متن مخفی هم ذخیره میشود. در حلقه، هر کاراکتر از متن مخفی به باینری تبدیل میشود و در `bin_secret` ذخیره میشود. باینری متن مخفی به یک رشته تبدیل میشه.

دوباره تصویر به بلوک‌ها ۶ در ۶ تقسیم میشود. یک ماتریس برای ذخیره واریانس بلوک‌ها تعریف میکنیم. در حلقه، واریانس هر بلوک محاسبه میشود که برای انتخاب بلوک‌های مناسب برای کدگذاری استفاده میشود. بلوک‌ها بر اساس واریانس مرتب می‌شن تا بلوک‌های با واریانس بالا اولویت بیشتری داشته باشند. تعداد کل بلوک‌ها و یک کپی از تصویر اصلی ایجاد میشود.

در حلقه بعد از آن، هر بلوک بر اساس اولویت انتخاب میشود. باینری متن مخفی به پیکسل‌های تصویر اضافه میشود و تصویر کدگذاری‌شده ساخته میشود. در نهایت، تصویر کدگذاری‌شده برگردانده میشود.

توضیح فانکشن دی کدینگ:

دقیقا مانند بخش کدینگ هست. قسمت‌های متفاوت کد رو توضیح خواهیم داد. یک آرایه خالی برای نگهداری پیام استخراج‌شده تعریف می‌شه. بعد از پیدا کردن بلوک با بیشترین واریانس مانند بخش کدینگ، پیکسل به پیکسل حرکت میکنیم و بیت آخر آن را ذخیره میکنیم و در هر مرحله ۵ بیت آخر را بررسی کرده و چک می‌کنیم که به عدد ۳۲ یا همان ۱۱۱۱ باینری که علامت است رسیدیم یا نه.

۵ بیت ۵ بیت از پیام خروجی را جدا کرده و با مپ ست مقدار عددی آن را پیدا میشود و پیام به صورت خروجی برگردانده میشود.

```

for i = 0:25
    Mapset{1, i+1} = char('a' + i);
    Mapset{2, i+1} = dec2bin(i, 5);
end

specialChars = ' .,!"';
for i = 0:5
    Mapset{1, i+27} = specialChars(i+1);
    Mapset{2, i+27} = dec2bin(i+26, 5);
end

```

(۲-۱) توضیحات بالا درج شده.

```

decoding.m x p2_1.m x p1.m x coding.m x +
function coded_pic = coding(secret, pic, Map)

    block_size = 6;
    [rows, cols] = size(pic);
    num_blocks_x = floor(cols / block_size);
    num_blocks_y = floor(rows / block_size);
    secret_len = length(secret);
    bin_secret = "";

    for idx = 1:secret_len
        current_char = secret(idx);
        char_position = find(contains(Map(1, :), current_char));
        if ~isempty(char_position)
            bin_secret = bin_secret + Map{2, char_position(1)};
        end
    end

    mat_bin_secret = char(bin_secret);

    row_sizes = ones(1, num_blocks_y) * block_size;
    col_sizes = ones(1, num_blocks_x) * block_size;
    all_blocks = mat2cell(pic, row_sizes, col_sizes);
    variance_blocks = zeros(num_blocks_y, num_blocks_x);

    for yy = 1:num_blocks_y
        for xx = 1:num_blocks_x
            block = all_blocks{yy, xx};
            block_msb = bitshift(block, -1);
            variance_blocks(yy, xx) = var(double(block_msb(:)));
        end
    end

```

```

end

[~, sorted_indices] = sort(variance_blocks(:), 'descend');
ranked_variance_block = zeros(size(variance_blocks));
ranked_variance_block(sorted_indices) = 1:numel(sorted_indices);

total_blocks = num_blocks_y * num_blocks_x;
pic_copy = pic;
current_pos = 1;

for w = 1:total_blocks
    [the_shape_row, the_shape_column] = find(ranked_variance_block == w);
    final_block = all_blocks{the_shape_row, the_shape_column};
    [block_shape_row_index, block_shape_column_index] = size(final_block);

    for c_r = 1:block_shape_row_index
        for c_c = 1:block_shape_column_index

            if current_pos <= length(mat_bin_secret)
                current_pixel = final_block(c_r, c_c);
                binarized_pixel = dec2bin(current_pixel, 8);
                binarized_pixel(end) = mat_bin_secret(current_pos);
                pic_copy((the_shape_row-1) * block_size + c_r, ...
                    (the_shape_column-1) * block_size + c_c) = bin2dec(binarized_pixel);
                current_pos = current_pos + 1;
            else
                break;
            end
        end
        if current_pos > length(mat_bin_secret)
            break;
        end
    end

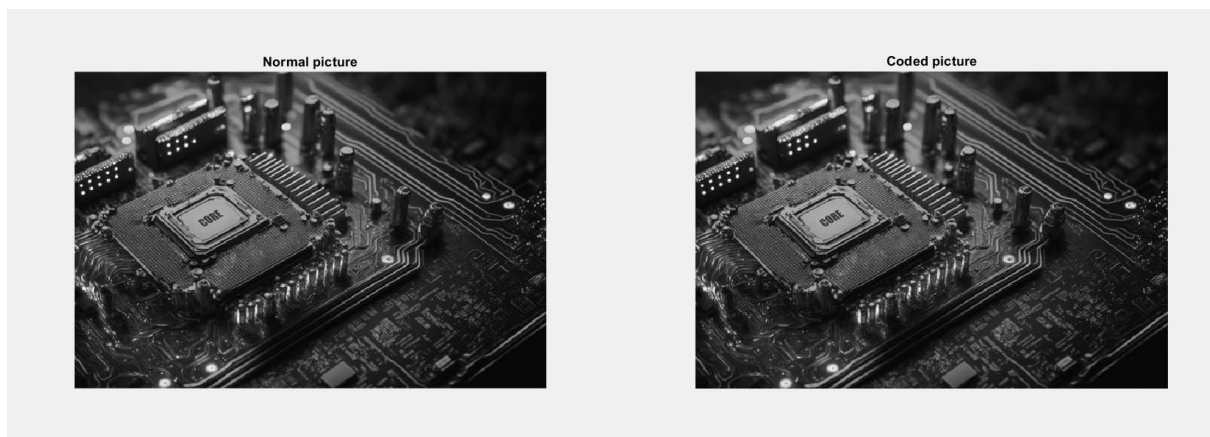
    (the_shape_column-1) * block_size + c_c = bin2dec(binarized_pixel);
    current_pos = current_pos + 1;
    else
        break;
    end
end
if current_pos > length(mat_bin_secret)
    break;
end
end
if current_pos > length(mat_bin_secret)
    break;
end
end
end

coded_pic = pic_copy;
end

```

(۳-۱)

نه مشخص نیست. چون بلوک های با بیشترین واریانس را برای مخفی کردن پیام برگزیدیم.



(۴-۱)

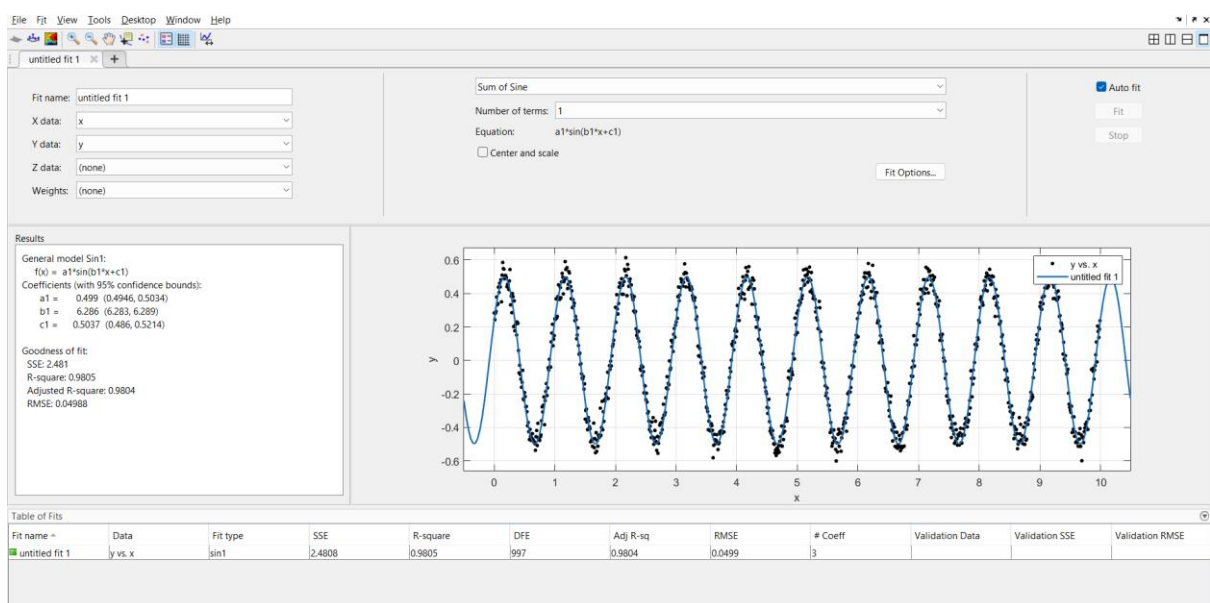
توضیحات داده شد. برای بلوک ساینز به طور پیشفرض همان ۶ که در کد اصلی بخش ۱ در نظر گرفته شده به عنوان استراتژی استفاده شد.

(۵-۱)

اگر به یک تصویر نویز اضافه شود، این تغییر می‌تواند کم‌ارزش‌ترین بیت‌ها را تحت تاثیر قرار دهد و موجب شود که پیام اصلی دیگر قابل استخراج نباشد. برای مقابله با این مشکل، می‌توان از تکنیک‌هایی استفاده کرد که علاوه بر ذخیره پیام اصلی، داده‌های کمکی دیگری نیز کدگذاری شوند. این داده‌ها به شناسایی وجود نویز در تصویر، تعیین بیت‌های آسیب‌دیده و اصلاح مقادیر صحیح آن‌ها کمک می‌کنند.

بخش دوم:

(۲-۱)



$$a0 = 0.499, w0 = 6.283, \text{phi}0 = 0.5037$$

(۲-۲)

<pre>a0 = 0.01:0.01:1; w0 = 0:pi/10:10*pi; phi0 = 0:pi/100:2*pi; numA0 = length(a0); numW0 = length(w0); numPhi0 = length(phi0); result = zeros(numA0, numW0, numPhi0); for i = 1:numA0 amplitude = a0(i); for j = 1:numW0 frequency = w0(j); for k = 1:numPhi0 phase = phi0(k); sineValue = amplitude * sin(frequency * x + phase); result(i, j, k) = sum((y - sineValue).^2); end end end [value, index] = min(result(:)); [i, j, k] = ind2sub(size(result), index); amplitude = a0(i); frequency = w0(j); phase = phi0(k); executionTime = toc;</pre>	<table><tr><td>a0</td><td>1x100 do...</td><td>800</td></tr><tr><td>amplitude</td><td>0.5000</td><td>8</td></tr><tr><td>executionTime</td><td>18.1877</td><td>8</td></tr><tr><td>frequency</td><td>6.2832</td><td>8</td></tr><tr><td>i</td><td>50</td><td>8</td></tr><tr><td>index</td><td>163650</td><td>8</td></tr><tr><td>j</td><td>21</td><td>8</td></tr><tr><td>k</td><td>17</td><td>8</td></tr><tr><td>numA0</td><td>100</td><td>8</td></tr><tr><td>numPhi0</td><td>201</td><td>8</td></tr><tr><td>numW0</td><td>101</td><td>8</td></tr><tr><td>phase</td><td>0.5027</td><td>8</td></tr><tr><td>phi0</td><td>1x201 do...</td><td>1608</td></tr><tr><td>result</td><td>100x101x...</td><td>162408...</td></tr><tr><td>sineValue</td><td>1x1000 d...</td><td>8000</td></tr><tr><td>value</td><td>2.5147</td><td>8</td></tr><tr><td>w0</td><td>1x101 do...</td><td>808</td></tr><tr><td>x</td><td>1x1000 d...</td><td>8000</td></tr><tr><td>y</td><td>1x1000 d...</td><td>8000</td></tr></table>	a0	1x100 do...	800	amplitude	0.5000	8	executionTime	18.1877	8	frequency	6.2832	8	i	50	8	index	163650	8	j	21	8	k	17	8	numA0	100	8	numPhi0	201	8	numW0	101	8	phase	0.5027	8	phi0	1x201 do...	1608	result	100x101x...	162408...	sineValue	1x1000 d...	8000	value	2.5147	8	w0	1x101 do...	808	x	1x1000 d...	8000	y	1x1000 d...	8000
a0	1x100 do...	800																																																								
amplitude	0.5000	8																																																								
executionTime	18.1877	8																																																								
frequency	6.2832	8																																																								
i	50	8																																																								
index	163650	8																																																								
j	21	8																																																								
k	17	8																																																								
numA0	100	8																																																								
numPhi0	201	8																																																								
numW0	101	8																																																								
phase	0.5027	8																																																								
phi0	1x201 do...	1608																																																								
result	100x101x...	162408...																																																								
sineValue	1x1000 d...	8000																																																								
value	2.5147	8																																																								
w0	1x101 do...	808																																																								
x	1x1000 d...	8000																																																								
y	1x1000 d...	8000																																																								

یک آرایه سه بعدی به نام **result** برای ذخیره نتایج محاسبات ایجاد می کنیم. ابعاد این آرایه برابر با تعداد مقادیر در **a0**، **w0** و **phi0** است. حلقه پس از آن روی سه متغیر تابع میچرخد و مقدار تابع در هر لوپ محاسبه میشود. سپس خطای برازش محاسبه و در آرایه **result** ذخیره می شود. کمترین مقدار خطا را از آرایه **result** پیدا می کند و همچنین اندیس آن را ذخیره می کند. اندیس خطای کمینه به مختصات سه بعدی تبدیل می شود تا بتوانیم مقادیر مربوط به دامنه، فرکانس و فاز را پیدا کنیم. مقادیر نهایی دامنه، فرکانس و فاز که کمترین خطا را ایجاد کرده اند، استخراج می شوند.

شرح نتایج:

زمان : ۱۸.۱۸ ثانیه ، امگا یا فرکانس : ۶.۲۸۳۲ ، دامنه ۰.۵ ، فاز: ۰.۵۰۲۷

با نتایج بخش قبل تقریباً برابر است.

(۳-۲)

summa - workspace - stop - pause - help

decoding.m * p1.m * coding.m * p2_3.m * +

1 clear all;

2 tic;

3

4 load('DataFit.mat');

5

6 func = zeros(10, 10, 20);

7 r = 1;

8 s = 1;

9 t = 1;

10

11 for a0 = 0.1 : 0.1 : 1

12 for w0 = 0 : pi : 10*pi

13 for phi0 = 0 : pi/10 : 2*pi

14 func(r, s, t) = sum((y - a0*sin(w0*x + phi0)).^2);

15 t = plus(t, 1);

16 end

17 s = plus(s, 1);

18 end

19 r = plus(r, 1);

20 t = 1;

21 s = 1;

22 r = plus(r, 1);

23 end

24

25 [value, index] = min(func(:));

26 [r, s, t] = ind2sub(size(func), index);

27

28 start_a = (r-1)*0.1;

29 stop_a = (r+1)*0.1;

30 start_w = (s-2)*pi;

31 stop_w = s*pi;

32

Name

Value

Bytes

Size

a0

0.5000

8

1x1

func

21x21x21...

74088

21x21x21...

index

2867

8

1x1

phi0

0.5027

8

1x1

r

11

8

1x1

s

11

8

1x1

second_time

0.1257

8

1x1

start_a

0.4000

8

1x1

start_phi

0.3142

8

1x1

start_w

3.1416

8

1x1

stop_a

0.6000

8

1x1

stop_phi

0.9425

8

1x1

stop_w

9.4248

8

1x1

t

7

8

1x1

value

2.5147

8

1x1

w0

6.2832

8

1x1

x

1x1000 d...

8000

1x1000 d...

y

1x1000 d...

8000

1x1000 d...

همان مراحل را انجام می دهیم با گامهای بزرگتر. **R, s, t** با کمک این متغیرها تابع مینیمم پیدا شده. سپس با گام های کوچکتر و بازه

کوچکتر حول مقادیر تخمین زده شده با گام بزرگ. یعنی ابتدا با گام بزرگ محدوده تقریبی را مشخص و با گام کوچک محدوده دقیق را مشخص می کنیم.

زمان: ۰.۱۲۵۷ ثانیه ، امگا یا فرکانس : ۶.۲۸۳۲ ، دامنه ۰.۵ ، فاز: ۰.۵۰۲۷

که با مقادیر بخش های قبل برابر و بسیار زمان کمتر شد.

(۴-۲)

سه متغیر $a0$ ، $\phi0$ و $w0$ رو به صورت سمبلیک تعریف می کنیم. اینکارو می کنیم که بتونیم مشتق گیری انجام بدیم و تابع خطا رو به صورت سمبلیک تحلیل کنیم. پس از اون تابع MSE (میانگین مربع خطا) رو تعریف کردیم. این تابع میزان خطای مدل سینوسی نسبت به داده های واقعی y رو محاسبه می کنه. بجای استفاده از سام از میانگین ضربدر ۱۰۰۰ استفاده کردم چون وقتی از سام استفاده شد غیرهمگرا میشد. در سه خط بعدش داریم مشتق تابع خطا رو نسبت به هر کدوم از پارامترها یعنی $a0$ ، $w0$ و $\phi0$ حساب می کنیم. این مشتق ها جهت شیب و تغییرات خطا رو نشون میدن، و بعداً توی الگوریتم بهینه سازی استفاده می شن. تو این بخش مشتق دوم تابع خطا رو نسبت به پارامترها حساب می کنیم. برای هندل کردن به صورت عددی از `matlabFunction` استفاده شد. و متغیرهای متناظرش رو تعریف کردیم.

پس از اون مقدار اولیه برای پارامترها رو مشخص کردیم. الگوریتم از این مقدارها شروع به بهینه سازی می کنه.

اگر نزدیک به مقادیر واقعی انتخاب کنیم مقادیر اصلی رو دقیق محاسبه میکنه ولی اگر نزدیک نباشه غیرهمگرا میشه و دقیق حساب نمیکنه (مثل چیزی که بعنوان آخرین تغییرات داخل کد متلب این بخش `p2_4` ذخیره شده). به صورت الگوریتم های محاسبات عددی یعنی کمینه اختلاف و بیشترین تعداد تکرار مجاز (واگرایی) این عمل تکرار میشه.

که در آخر پس از محاسبه بردار گرادیان و ماتریس انحنا و انجام محاسبات و ... و اتمام حلقه برنامه به پایان میرسد. که نتیجه به صورت برجسته در بالا ذکر شده است.

نتیجه نادرست به دلیل مقدار اولیه نادرست یعنی واگرایی:

```
Command Window

5.3725e-07

10.0561

0.1956

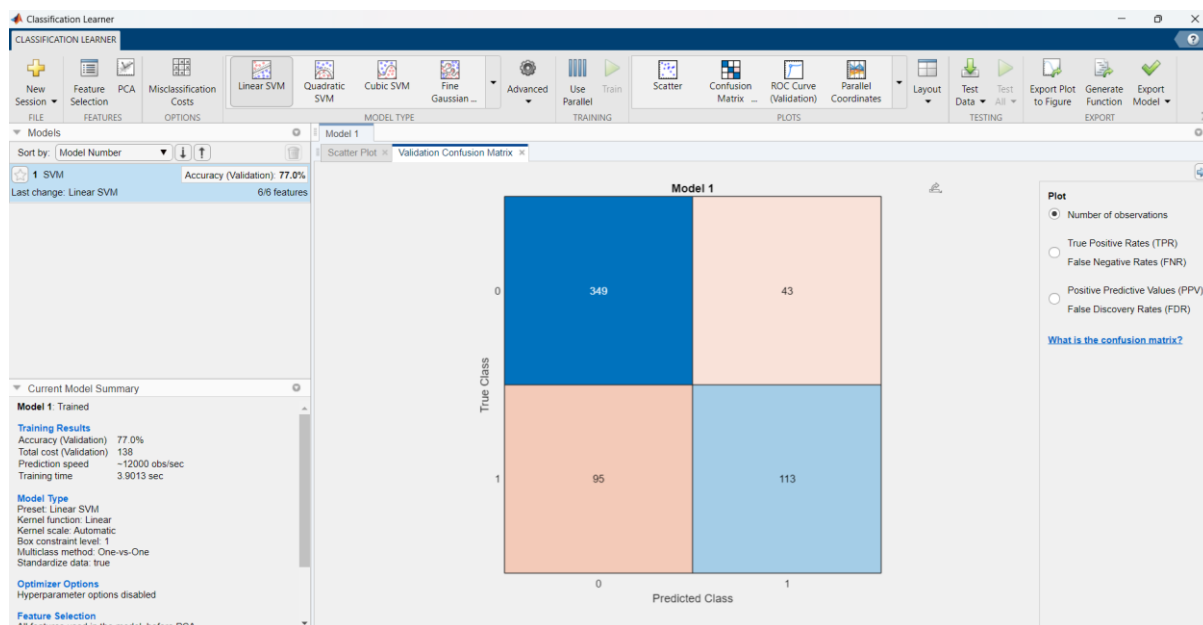
Elapsed time is 24.520586 seconds.
fx >>
```

(۲-۵) همیشه همگرا نمیشود توضیح در بخش قبل گفته شد: پس تابع دارای مینیمم های محلی است.

بخش سوم:

(۳-۱)

دقت: ۷۷ درصد



(۲-۳)

گلوکز: ۷۳.۳ و انسولین: ۶۵.۳ و پوست ضخیم: ۶۵.۳ و سن: ۶۵.۳ و فشار خون: ۶۵.۳ و bmi: ۵۹

درصد

گلوکز بیشترین ارتباط را دارد.

(۳-۳)

استفاده از کد راهنما:

دقت: ۷۷.۵ درصد


```
decoding.m x p1.m x coding.m x p2_3.m x p2_4.m x p3.m x
1 close all;
2 clc;
3
4 train_data = readtable('diabetes-training.csv');
5
6
7 load('TrainedModel.mat');
8
9 [trainingfit,~] = TrainedModel.predictFcn(train_data);
10 training = table2array(train_data(:,size(train_data,2)));
11 passed_train = 0;
12
13 for i = 1:size(train_data,1)
14     if training(i) == trainingfit(i)
15         passed_train=plus(passed_train,1);
16     end
17 end
18 accuracy = (passed_train/size(train_data,1))*100;
19 disp(accuracy);
```

(۳-۴)

دقت: ۷۸ درصد

```
clear all;

load('TrainedModel.mat');

valid_data = readtable('diabetes-validation.csv');

[fiting,~] = TrainedModel.predictFcn(valid_data);
fit = table2array(valid_data(:,size(valid_data,2)));
correct_fit=0;
for i = 1:size(valid_data,1)
    if fit(i) == fiting(i)
        correct_fit=plus(correct_fit,1);
    end
end
accuracy = (correct_fit/size(valid_data,1))*100;
disp(accuracy);
```

بخش چهارم:

```

decoding.m x p1.m x coding.m x p2_3.m x p2_4.m x p3.m x p3_4.m x p4.m x +
1 clear all;
2 td = 0.01;
3 t = 0:td:1;
4
5 a = ramp(t) - ramp(t-1) - heaviside(t - 1);
6 b = ramp(-t+1) - ramp(-t) - heaviside(-t);
7 c = 0.5 * (heaviside(t) - heaviside(t - 1));
8
9 figure;
10 xlim([0 2])
11 xlim([0 0.3])
12 subplot(4,3,1);
13 plot(t , a);
14 title("a");
15 subplot(4,3,2);
16 plot(t , b);
17 title("b");
18 subplot(4,3,3);
19 plot(t , c);
20 title("c");
21 y_1 = conv(a, a) * td;
22 y_2 = conv(a, b) * td;
23 y_3 = conv(a, c) * td;
24 y_4 = conv(b, a) * td;
25 y_5 = conv(b, b) * td;
26 y_6 = conv(b, c) * td;
27 y_7 = conv(c, a) * td;
28 y_8 = conv(c, b) * td;
29 y_9 = conv(c, c) * td;

```

```

decoding.m x p1.m x coding.m x p2_3.m x p2_4.m x p3.m x p3_4.m x p4.m x +
27 y_7 = conv(c, a) * td;
28 y_8 = conv(c, b) * td;
29 y_9 = conv(c, c) * td;
30 t_conv = 0:0.01:(length(y_1)-1) * td;
31 subplot(4,3,4);
32 plot(t_conv , y_1 );
33 title("conv(a, a)");
34 subplot(4,3,5);
35 plot(t_conv , y_2);
36 title("conv(a, b)");
37 subplot(4,3,6);
38 plot(t_conv , y_3 );
39 title("conv(a, c)");
40 subplot(4,3,7);
41 plot(t_conv , y_4);
42 title("conv(b, a)");
43 subplot(4,3,8);
44 plot(t_conv , y_5 );
45 title("conv(b, b)");
46 subplot(4,3,9);
47 plot(t_conv , y_6);
48 title("conv(b, c)");
49 subplot(4,3,10);
50 plot(t_conv , y_7 );
51 title("conv(c, a)");
52 subplot(4,3,11);
53 plot(t_conv , y_8);
54 title("conv(c, b)");

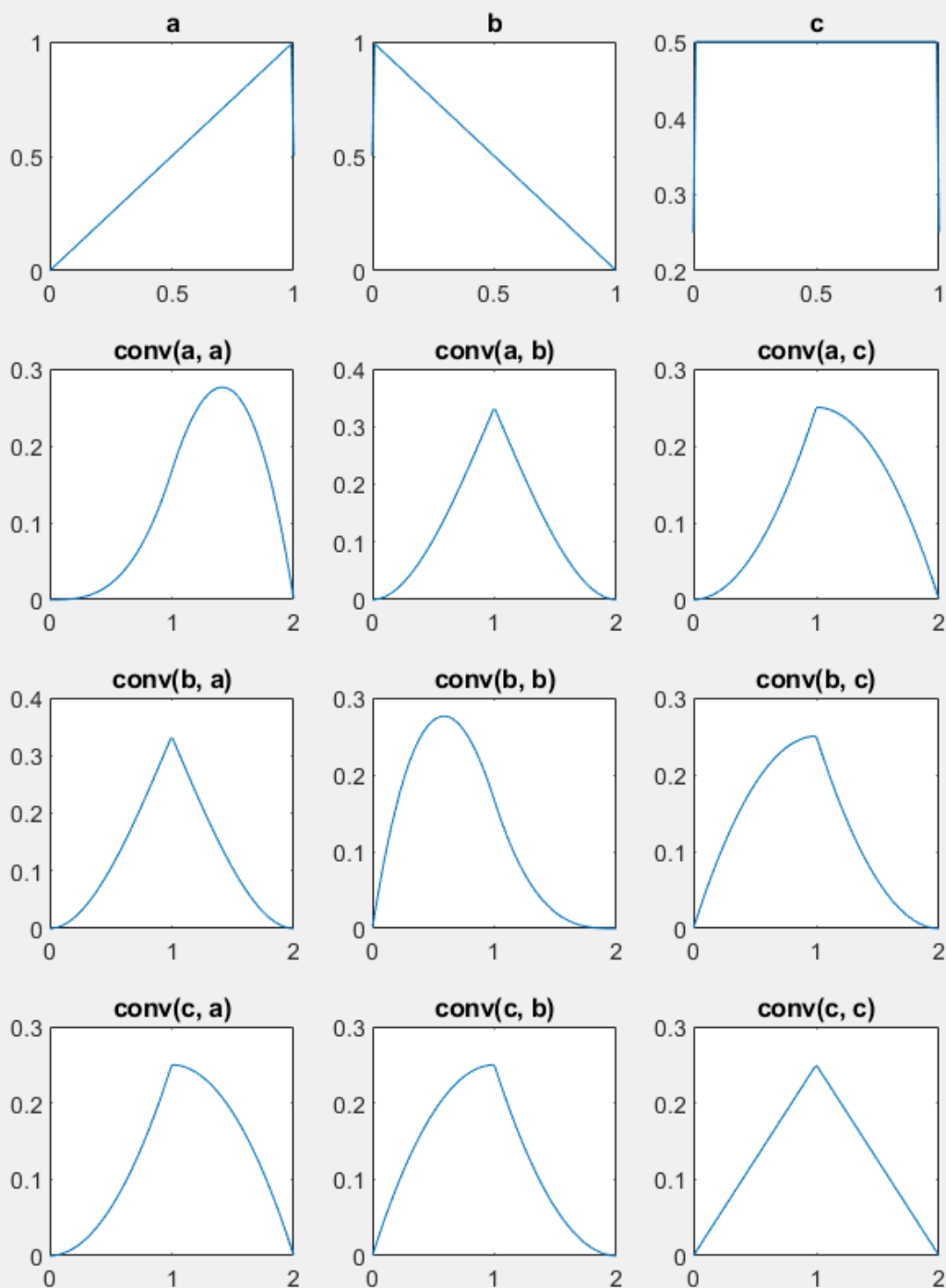
```

```

44 plot(t_conv , y_5 );
45 title("conv(b, b)");
46 subplot(4,3,9);
47 plot(t_conv , y_6);
48 title("conv(b, c)");
49 subplot(4,3,10);
50 plot(t_conv , y_7 );
51 title("conv(c, a)");
52 subplot(4,3,11);
53 plot(t_conv , y_8);
54 title("conv(c, b)");
55 subplot(4,3,12);
56 plot(t_conv , y_9 );
57 title("conv(c, c)");
58
59
60 function y = ramp(t)
61     y = zeros(size(t));
62     for i = 1:length(t)
63         if t(i) >= 0
64             y(i) = t(i);
65         else
66             y(i) = 0;
67         end
68     end
69 end
70
71

```

تصاویر کانولوشن ها :



با توجه به نمودار های بالا بعضی از آنها با هم برابرند

